Lecture 12

局部搜索

12.1 基本思想

假如我们在一片山地中，想要走到海拔最低的位置，最自然的办法是看看附近哪里更低，就往哪里走，直到四周都没有下坡路。把这一想法用于优化问题，就得到了**局部搜索（local search）**：先找一个可行解，每次对它作一点改动，只要结果更好就继续，直到这样的改动都不能改善结果。

当然，很显然的问题是如果这座山是一个山顶一个山谷这样连绵起伏的，我们很有可能走到的只是一个局部的山谷，也就是局部的最低点，并非整座山脉的最低点。图 12.1 中，要从左边山谷走到右边更深的山谷，必须先上坡。

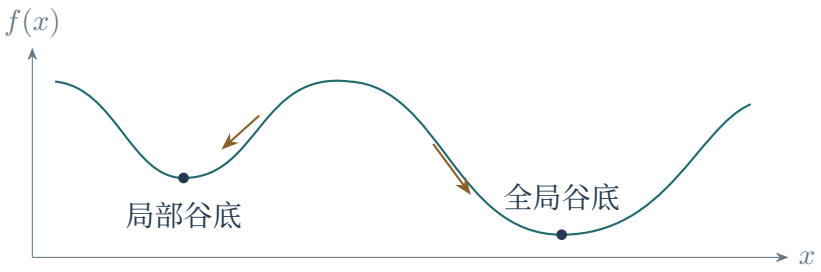


图 12.1 沿着下坡走，可能停在左边的山谷；到右边更深的山谷需要先上坡。

既然局部搜索可能停在较差的解，为什么还要使用它呢？一个原因是全局最优可能太难找。前面已经见过许多 NP 困难问题，枚举所有可行解又往往需要指数时间。因此，我们可以先从容易得到的解出发，通过局部改进寻找更好的解。补充材料第 12.6.1 节还会用一个具体问题说明寻找全局最优解的困难。

12.1.1 自然科学中的局部搜索

前面我们把局部搜索比作沿山坡向下走，其实物理系统也会出现类似的过程。把一个小球放在有阻力的山坡上，它可能经过一段滚动和振荡，最终停在某个谷底。这个位置附近已经没有更低的地方，但远处仍可能有更深的山谷。如果把小球的位置看成当前解，把高度看成要最小化的目标，就很容易理解局部搜索为什么会停在局部最优。后面介绍模拟退火时，还会借用物理系统在不同温度下的行为，让搜索有机会离开这样的谷底。

计算化学中也有直接需要局部优化的问题。例如，我们想求一个分子的稳定结构，可以用各个原子的位置表示一种候选结构，并计算它的能量。从一个初始结构出发，逐步调整原子的位置，使能量降低，直到附近的小幅调整都无法继续降低能量，就找到了一个局部最小值。这里，原子的位置对应

解，位置的微小调整对应局部改动，能量对应目标函数。不同的初始结构可能得到不同的结果，正如从山坡的不同位置出发，可能落到不同的谷底。

生物的变异与选择则启发了另一类优化方法，称为进化计算（evolutionary computation）。它同时保留一批候选解，将待解决的问题看作环境、每个候选解看作一个个体，并用目标值衡量个体的适应性。算法先生成一批个体，称为种群（population）。每一轮从中选出亲代，通过修改亲代或组合多个亲代产生后代，再从原有个体和后代中选出下一代。反复进行这个过程，希望种群中出现越来越好的解。它与局部搜索的联系在于，许多新解也是通过修改已有解得到的。不过它可以同时保留多个解，还可以重组两个解的不同部分，搜索方式更为丰富。要把这个想法用于具体问题，首先得决定怎样表示一个个体，然后才能规定怎样修改它。后续会通过一个背包例子说明其中的基本做法。

此外，我们还需要选择亲代和下一代。可以让较好的个体更容易被选为亲代，也可以规定目前最好的个体一定进入下一代。前者希望通过较好的亲代产生更好的后代，后者则保证已经找到的最好结果不会丢失。这里的“好”由评估函数决定，所以评估函数必须反映真正的目标。

一个非常经典的笑话就来源于评估函数设计不当：如果要让一只电子狼在尽可能短的时间内避开障碍，追上一只仿生人梦到的电子羊，那么一只最厉害的电子狼可能是直接撞向障碍物的一只。它花费了最短的时间，抵达了狼生的终点。如果程序只比较耗时，却没有要求成功抓到羊，就会把这种失败当成好结果。算法确实找到了耗时很短的行为，但这显然偏离了我们的本意。

类似的变异与筛选也用于软件的模糊测试（fuzzing）。程序从已有测试输入出发，修改其中一部分，再把新输入交给被测程序。如果它触发了以前没有经过的执行路径，就可以保留下来，继续生成新的测试。例如，测试 XML 处理程序时，可以保留标签结构、只修改内容，让输入有机会进入更深的处理过程，也可以破坏结构来检查程序怎样处理错误。

举个例子，我们希望通过进化计算来解决 0-1 背包问题。一个个体可以表示为 n 位二进制字符串，第 i 位对应第 i 件物品。最直接的解释是，位为 1 就选这件物品，但这样可能产生超重的选择。另一种办法是按编号依次检查物品，只有这一位为 1、且背包还装得下时才选入。这样，每条字符串都能得到一个可行的物品集合。后一种办法也有一个特点：不同字符串可能得到相同的选择。例如，前几件物品已经把背包装满，后面的位无论是 0 还是 1 都不会改变结果。基于这一编码，我们可以计算任意字符串对应的物品选择的总价值，作为它的适应性。有了字符串及其对应的价值，就可以规定怎样产生下一代。例如我们可以每轮随机抽取两个个体，选价值较大的一个作为亲代，复制后随机翻转一位，得到一个后代。如果后代不比种群中最差的个体更差，就用它替换最差个体。后代只来自一个亲代，因此这里采用的是“无性生殖”。若想结合两个亲代，也可以按预先规定的概率，从它们各自的对应位置选取比特来组成后代。

12.1.2 局部搜索的范式

要把局部搜索写成算法，首先需要明确“附近”包括哪些解。对于可行解 S，我们可以规定一步允许作哪些修改。所有通过这些修改得到的可行解，组成 S 的**邻域（neighborhood）**，记为

$$N(S) = \{S' \mid S' \text{ 是对} S \text{ 作一次允许的改动后得到的可行解}\}.$$

邻域由我们根据问题特点来设计。例如，背包可以允许替换一件物品，旅行商问题可以允许改变路线中的两条边。规定邻域后，就从一个可行解开始，检查它的邻居是否有更好的目标值。如果有，就换成一个更好的邻居，再重复检查。如果所有邻居都不能严格改善目标值，就得到了这个邻域下的**局部最优解**。下面以最小化问题为例，每次选取目标值最小的邻居。当然也可以找到第一个更好的邻居就继续搜索，两种选择可能经过不同的解。

算法 12.1　LocalSearch：寻找局部最优解

输入： 有限可行域中的初始解 S，目标函数 f，邻域 $N(S)$ 的枚举方法
输出： 关于 N 的局部最优解

```
while true do
    next ← S
    for 按固定顺序枚举 S' ∈ N(S) do
        if f(S') < f(next) then
            next ← S'
        end if
    end for
    if next = S then
        return S
    end if
    S ← next
end while
```

把每个可行解画成一个顶点，每次能够严格改善目标值的移动画成一条有向边，就得到图 12.2。局部搜索沿着箭头走，走到没有出边的顶点时停止。因为目标值每次都严格下降，沿箭头不可能回到已经访问过的解，所以图中没有有向环。因为可行解只有有限个，算法一定会停下来。然而算法能终止并不代表算法很快，因为可行解可能有指数多个。需要强调的是，算法停止有两个重要的条件：(1) 可行解有限：例如最小化 $f(x)=x^2$，从 $x=1$ 开始，每次把 x 减半，函数值始终下降，却永远不会在有限步到达 0，因为此时可行解无限；(2) 目标值严格下降：如果允许在等值解之间移动，就需要额外防止来回循环，或者直接规定最多搜索多少步。

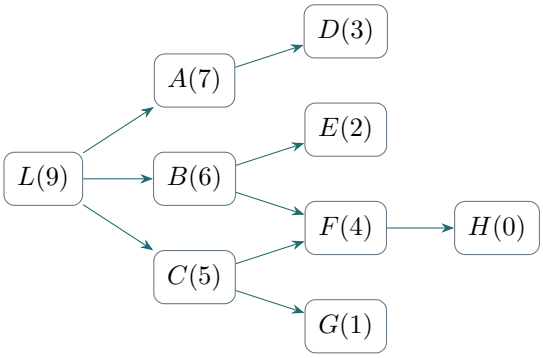


图 12.2　括号内为目标值，沿箭头移动会使目标值严格下降；没有出边的四个顶点是局部最优解。

12.2　顶点覆盖问题

12.2.1　删除一个顶点

首先我们以顶点覆盖为例，看看怎样选择初始解、规定邻域，以及这种选择会得到怎样的结果。给定无自环的无向图 $G=(V,E)$，我们希望选出尽可能少的顶点，使每条边至少有一个端点被选中，这就是顶点覆盖问题。

习题 11.6 已经讨论过一个 2-近似算法。现在尝试局部搜索。既然目标是减少所选顶点数，最自然的改动就是删掉一个顶点，同时保证剩下的集合仍然覆盖全部边。初始解可以直接取 $S = V$，然后不断尝试删除。怎样判断顶点 u 能不能删呢？自然地，不与 u 相连的边不受影响，与 u 相连的每条边 (u, v)，删除 u 后都必须由另一个端点 v 覆盖。因此，只有 u 的所有邻居都还在 S 中时，才能删除 u。

事实上，我们只需要遍历一次所有顶点就可以完成局部搜索，这是因为此前遍历过的顶点如果不能删除，则说明其某个邻居 v 已不在集合中。后面只会继续删点，v 不会再回来，所以 u 以后也不能删。因此总时间复杂度为 $O(|V| + |E|)$。

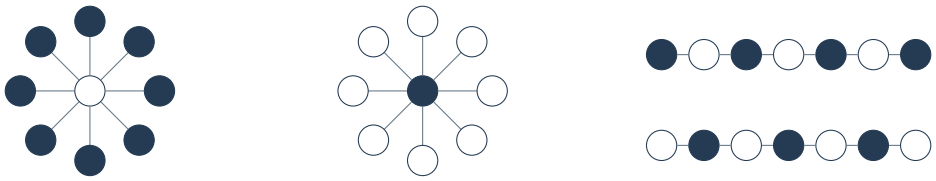


图 12.3 顶点覆盖局部搜索的几个例子，其中实心圆表示选择的顶点。

这个算法得到的覆盖有多好呢？如果图没有边，所有顶点都能删掉，最后得到最优的空覆盖。但图 12.3 中的星形图就很不一样。如果先删除中心，外围八个顶点将一个都不能删，结果大小为 8。如果先删除外围顶点，最后只需保留中心，大小为 1。把外围顶点数增加到任意大的 r，两种结果就是 r 与 1，所以单纯的删点法没有固定的近似比。最右图中的路径也说明，同一张图上可以有大小不同的局部最优覆盖。

我们可以从两个方向改进。一种是先用已有的 2-近似算法得到初始覆盖，再作合法删除，这样覆盖只会变小，原来的近似保证仍然成立。另一种是允许搜索暂时走向较差的解，同时争取离开当前的局部最优。下面讨论后一种办法。

12.2.2 Metropolis 算法与模拟退火

还是看星形图。如果当前覆盖包含全部叶子、却没有中心，想改成只选中心，就必须先把中心加回来。虽然这一步让覆盖变大，但此后就能把叶子逐个删去。

Metropolis 等人于 1953 年提出的方法来自统计物理。把每个可行解看成一个状态 S，把要最小化的目标值记为能量 $E(S)$。温度为 $T > 0$ 时，Boltzmann 分布给各状态的概率为

$$\pi_\theta(S) = \frac{e^{-E(S)/\theta}}{Z_\theta}, \qquad Z_\theta = \sum_{R \in \Omega} e^{-E(R)/\theta}, \qquad \theta = kT,$$

其中 Ω 是有限状态集合，k 是玻尔兹曼常数，分母使概率之和等于 1。比较两个状态时，能量较低的状态概率较大。温度高时，两者的概率差别较小；温度低时，分布会更偏向低能量状态。

为了模拟这种偏好，算法每轮随机提出一个新状态 S'。如果 $E(S') \leqslant E(S)$，就接受它；如果能量增加了 $\Delta > 0$，则以概率 $e^{-\Delta/\theta}$ 接受。拒绝时保留当前状态，继续下一轮。这样，较差的解也有机会被接受，而且变差越多或温度越低，接受的概率就越小。

还需要规定怎样提出新状态。记 $q(S, S')$ 为当前处于 S 时提出 S' 的概率，本节要求 $q(S, S') = q(S', S)$，称为**对称提议**。例如，从一组固定的可逆操作中随机选择，每种操作与它的逆操作被选中的概率相同，就能满足这一要求。下面还让每轮以 $1/2$ 的概率保持不动，避免状态只按固定周期往返。

算法 12.2 Metropolis 算法

输入： 有限非空状态集合 Ω，能量函数 $E:\Omega\to\mathbb{R}$，初态 $S\in\Omega$
输入： 对称提议概率 q 的抽样方法，步数 $B\geqslant 0$，正温度序列 $\theta[1..B]$
输出： 访问过的能量最低的状态 $best$

$best \leftarrow S$
for $t=1$ **to** B **do**
  if 独立公平硬币为正面 **then**
    按概率 $q(S,\cdot)$ 抽取候选状态 $S'\in\Omega$
    $\Delta \leftarrow E(S')-E(S)$
    if $\Delta\leqslant 0$ 或独立的 $U\sim\mathrm{Unif}[0,1)$ 满足 $U<e^{-\Delta/\theta[t]}$ **then**
      $S\leftarrow S'$
      if $E(S)<E(best)$ **then**
        $best\leftarrow S$
      end if
    end if
  end if
end for
return $best$

用于顶点覆盖时，Ω 是全部可行覆盖，$E(S)=|S|$。每次从全部顶点中均匀选取一个，尝试加入或删除它。若结果不再是覆盖，就以原状态作为候选。两个只差一个顶点的可行覆盖之间，正反方向都以 $1/|V|$ 的概率提出修改，因此提议对称。任意覆盖都能先加点得到 V，再删点得到另一个覆盖，所以所有状态也能互相到达。若 V 为空，直接返回空覆盖即可。

先令温度保持不变，即 $\theta[t]=\theta$。下面解释上述接受规则为什么对应 Boltzmann 分布。记 $P(S,S')$ 为一步之后从 S 到达 S' 的概率。对于任意两个不同状态 S,S'，一轮中实际提出这次修改的概率为 $q(S,S')/2$。由提议的对称性，

$$\begin{aligned}\pi_\theta(S)P(S,S') &= \frac{q(S,S')}{2Z_\theta}e^{-E(S)/\theta}\min\{1,e^{-(E(S')-E(S))/\theta}\}\\ &= \frac{q(S,S')}{2Z_\theta}\min\{e^{-E(S)/\theta},e^{-E(S')/\theta}\}\\ &= \pi_\theta(S')P(S',S).\end{aligned}$$

等式左边是在当前分布为 π_θ 时，从 S 转移到 S' 的概率，右边是反向转移的概率。两者相等，称为**细致平衡（detailed balance）**。此外，不能直接互相转移的两个状态，两边都是 0，而若 $S=S'$，等式两边本来就是同一项。把所有可能的出发状态 S 加起来，就得到下一轮位于 S' 的概率：

$$\sum_S \pi_\theta(S)P(S,S')=\pi_\theta(S')\sum_S P(S',S)=\pi_\theta(S').$$

因此，如果某一轮状态服从 π_θ，下一轮仍服从这个分布，称它为**平稳分布（stationary distribution）**。

实际运行通常从一个指定的状态开始，并不会一开始就服从这个分布。不过，运行足够久以后，仍有下面的结论。

定理 12.1. 设 Ω 有限且非空，E 在每个状态上都取有限实数值，提议概率 q 对称，且任意两个状态都能通过若干次正概率提议互相到达。固定 $\theta > 0$，无限运行上述 Metropolis 算法，则从任意初态出发，当前状态的分布都收敛到 π_θ。对每个状态 R，前 t 轮处于 R 的比例以概率 1 收敛到 $\pi_\theta(R)$。

由于温度为正，每次提议的接受概率都大于零，所以提议能连通的状态在实际运行中也能互相到达。算法又保证每个状态至少以 $1/2$ 的概率保持不动，结合上面证明的平稳性，就能推出收敛结论。完整证明见补充材料 12.6.2。

选择新状态的方法会影响这个结论。例如，若每次只从当前状态的合法邻居中均匀选择，邻居较多的状态就更容易被访问。考虑三个等能量状态组成的路径 a–b–c，从 a 提出的候选一定是 b，从 b 却只有一半概率提出 a。这样会得到长期访问比例为 $1/4, 1/2, 1/4$，而我们希望的 Boltzmann 分布在三个状态上都是 $1/3$。一般地，若提议概率 $q(S, S')$ 不对称，可以采用 Metropolis–Hastings 接受率

$$\min\left\{1, e^{-(E(S')-E(S))/\theta}\frac{q(S', S)}{q(S, S')}\right\},$$

用概率比补偿两个方向的差异。对于两个不同状态，将这个接受率代入转移概率，得到

$$\pi_\theta(S)P(S, S') = \frac{1}{2}\min\{\pi_\theta(S)q(S, S'), \pi_\theta(S')q(S', S)\}.$$

其中 $1/2$ 来自算法每轮尝试修改的概率。交换 S 与 S' 后，右边不变，因此正反方向仍满足细致平衡，π_θ 也就仍是平稳分布。

此外，Metropolis 算法的温度不必始终固定。我们可以开始时先用较高温度，让算法比较容易接受暂时变差的修改，以便离开较差的局部最优。此后逐渐降低温度，使算法更倾向于保留较好的解。这就是**模拟退火（simulated annealing）**的基本思想，名称来自物理系统的缓慢降温过程。一种常见的做法是，每运行 b 步就把温度乘以 $0 < \alpha < 1$。给定初温 $\theta_0 > 0$，相应的温度序列为 $\theta[t] = \theta_0\alpha^{\lfloor (t-1)/b \rfloor}(1 \leqslant t \leqslant B)$。将它代入 Metropolis，就得到一个有限步的模拟退火算法。如果一开始就用很低的温度，接受能量增加 $\Delta > 0$ 的修改的概率 $e^{-\Delta/\theta}$ 会很小，算法可能等待很久也无法离开当前的局部最优解，降温太快也会遇到同样的问题。因此，温度安排会直接影响搜索效果，实现时需要根据问题特点和可用时间来选择。

12.3 旅行商问题

顶点覆盖例子说明，局部最优可能与全局最优相差很远。不过，如果从已有近似保证的解开始，只作严格改进，原来的保证就能保留下来。旅行商问题正好可以这样处理：上一讲给出了度量 TSP 的 2-近似算法，下面尝试在它得到的路线附近继续寻找更短的路线。以下讨论顶点数 $n \geqslant 3$ 的情形。

12.3.1 动态规划算法

在正式讨论局部搜索前，先补全一个之前遗漏的讨论。旅行商问题虽然是 NP 困难的，但输入不大时，我们仍可能希望求出精确的最优解。当然，直接枚举访问顺序需要考虑阶乘多个排列，而使用动态规划可以合并其中的重复计算，得到相对高效的精确算法。

仍然用一个思维实验开始：假如上帝给了你一个最优回路，我们只观察它最后一跳的选择。给顶

点编号 $1,\dots,n$，固定从 1 出发。最后一定从某个 $j \neq 1$ 回到 1，因此

$$\text{最优回路长度} = \min_{j\in\{2,\dots,n\}} \{\text{从 1 出发、访问其余各点一次并停在}j\text{ 的最短路程} + w(j,1)\}.$$

接下来还可以继续看，到达 j 之前最后访问的是谁。不过，只记最后一个顶点还不够，因为我们还得知道哪些点已经访问过，才能避免重复访问。因此，子问题同时记录已访问的顶点集合和当前终点。设 $S \subseteq \{2,\dots,n\}$，$j \in S$，用 $D[S,j]$ 表示从 1 出发，恰好访问 S 中各点一次并停在 j 的最短路程。若 S 只有一个点，直接从 1 走过去即可：$D[\{j\},j] = w(1,j)$。若 $|S| \geqslant 2$，设到达 j 前位于 k，此前恰好访问了 $S \setminus \{j\}$。枚举所有可能的 k，得到

$$D[S,j] = \min_{k\in S\setminus\{j\}} \{D[S\setminus\{j\},k] + w(k,j)\}.$$

这就是 Held–Karp 算法，下面分析它的时间复杂度。由于状态总数为 $(n-1)2^{n-2}$，并且计算每个状态对应的值需要 $O(n-2)$ 的时间，因此总时间为 $O(n^2 2^n)$。相比之下，固定起点后，直接枚举其余城市的访问顺序，共有 $(n-1)!$ 种排列。每条路线用 $O(n)$ 时间计算总路程，合计需要 $O(n!)$ 时间。注意若只取阶乘中最大的约一半因子，就有 $n! \geqslant (n/2)^{n/2}$。取对数后，$\log(n!) \geqslant \frac{n}{2}\log\frac{n}{2}$，而 $\log(n^2 2^n) = n + 2\log n$，前者的增长速度更快，所以当 n 足够大时，阶乘远大于 $n^2 2^n$。

12.3.2　2-变换

接下来回到局部搜索。对于一条旅行路线，怎样修改才能得到一条邻近的路线呢？如果只删一条边，回路会变成一条路径，要重新连成回路，只能把原边加回去。因此至少要同时改变两条边，才可能得到不同的无向回路。

删去两条不相邻的边后，回路分成两段路径。保留这两段路径，把端点换一种方式连接，便可以得到新回路，称为 **2-变换（2-change 或 2-opt move）**。图 12.4 中，删去 $(v,w),(u,x)$，再加入 $(v,x),(u,w)$，仍能形成一条访问所有顶点的回路。如果连接 $(v,u),(w,x)$，会让两段路径各自成环，得到的就不是合法的旅行路线。

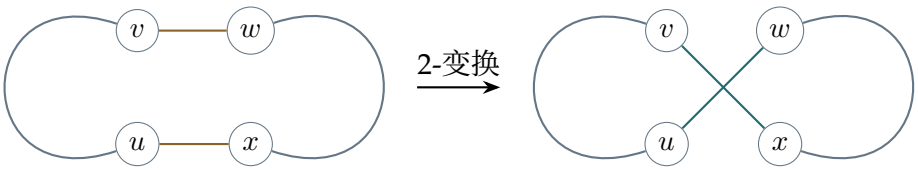


图 12.4　左右弯曲的线分别表示两段完整路径。合法重接仍形成一个环；不能各自封闭两段路径。

将当前访问顺序存为 $p[1..n]$，并约定 $p[n+1] = p[1]$。选择两条不相邻的边 (p_i,p_{i+1}) 和 (p_j,p_{j+1})，其中 $i<j$。将它们改成 (p_i,p_j) 和 (p_{i+1},p_{j+1})，相当于把中间一段 $p[i+1..j]$ 的访问顺序反转。由于图是无向的，中间各条边反向后长度不变，所以总路程的增量只取决于被替换的四条边：

$$\Delta = w(p_i,p_j) + w(p_{i+1},p_{j+1}) - w(p_i,p_{i+1}) - w(p_j,p_{j+1}).$$

只要 $\Delta < 0$，就可以接受这次变换。局部搜索算法每轮需要检查 $O(n^2)$ 对边，选取缩短最多的一次变换，因此若执行了 t 次变换，连同最后一次检查没有更优变换，总时间为 $O(n^2(t+1))$。如果不限

制步数，路线长度每次严格下降，而不同路线只有有限条，算法一定会在某个 2-opt 局部最优解处停止。不过，这并不能保证变换次数为多项式，因此实现时可以设置步数上限。

图 12.5 给出了五个城市的例子，各边权重为

边	ab	bc	be	ac	ad	bd	cd	ce	de	ae
权重	1	2	3	4	5	6	7	8	9	10

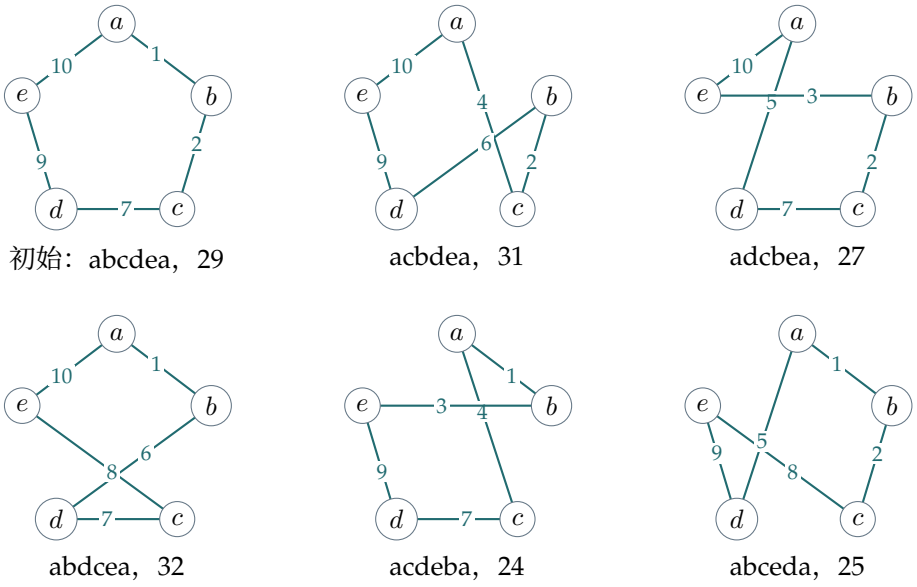


图 12.5 初始回路及其五个 2-opt 邻居，每个邻居都由初始回路改变两条边得到。

初始回路 a, b, c, d, e, a 的长度为 29。它的五个 2-opt 邻居长度分别为 $31, 27, 32, 24, 25$，所以本轮选择 a, c, d, e, b, a，将长度减少 5。

12.4 Hopfield 神经网络的稳定构型

前面的局部搜索都是为了尽量接近全局最优，接下来却会遇到一种情况：找到局部最优，就已经完成了任务。这一例子来自 Hopfield 神经网络。网络中的每个神经元会根据相邻神经元的状态调整自己，我们希望找到一个谁都不再需要调整的状态，称为稳定构型。

John J. Hopfield 于 1982 年提出了经典的反馈神经网络模型，并与 Geoffrey E. Hinton 共同获得了 2024 年诺贝尔物理学奖。本讲只讨论稳定构型的寻找，可以把它理解为下面的带权图问题。

12.4.1 稳定构型的定义

用无自环的无向图 $G = (V, E)$ 表示网络，每个顶点表示一个神经元，每条边表示两者之间的连接。顶点 u 的状态为 $s_u \in \{-1, 1\}$，所有顶点的状态合在一起，称为一个**构型（configuration）**。

边权可以为正，也可以为负。本讲规定，正权边希望两个端点状态相反，负权边希望两个端点状态相同。满足这一要求的边称为**好边**，也就是 $w_{uv} s_u s_v < 0$，其余称为**坏边**。零权边不影响计算，可以直接删去。有些物理文献把连接强度记为 $J_{uv} = -w_{uv}$，因而正负号的含义会与这里相反。

一个顶点通常连接多条边，因此无法只看其中一条来决定是否改变自己的状态。我们按权重的绝对值衡量各条边的重要程度，记与 u 相连的好边、坏边的权重绝对值之和分别为 g_u, b_u。如果 $g_u \geqslant b_u$，就称 u **满意**，否则称它**不满意**。如果我们翻转 u 的状态，关联的好边与坏边会全部交换，所以当坏边的总权重更大时，翻转才会使好边总权重增加。

为了方便计算，记

$$h[u] = \sum_{v \in Adj[u]} w_{uv} s_u s_v = b_u - g_u.$$

每条坏边在和中贡献正的绝对权重，好边则贡献负的绝对权重，因此 u 满意等价于 $h[u] \leqslant 0$。当所有顶点都满意时，就称整个构型为**稳定构型**。我们要解决的问题是，给定网络，怎样找到这样的构型。

例如图 12.6 的初态中，所有顶点都为 $+1$。对中心 c，边 ac 的正权重为 8，两端状态相同，因此它是坏边。另外三条负权边都是好边，权重绝对值之和为 $5+1+1=7$。所以 c 不满意，翻转后便会满意。不过它的邻居也受到影响，还需要继续检查。图中随后依次翻转 d, e，最终所有顶点都满意。

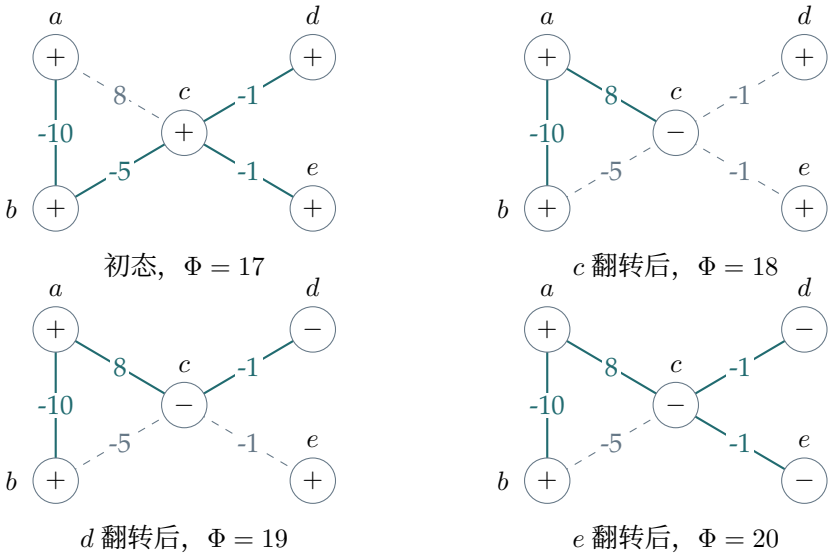


图 12.6　依次翻转 c, d, e。实线为好边，虚线为坏边，圆内标出状态。四幅图的好边权重和依次为 $17, 18, 19, 20$。

12.4.2　局部搜索算法

从上面的例子就可以得到一个自然的算法。我们可以把只翻转一个顶点得到的构型作为邻居，从任意初态开始，只要还有不满意顶点，就选一个翻转，直到所有顶点都满意。这一算法称为状态翻转算法（state-flipping algorithm）。

算法的复杂度分析是简单的。首先我们要初始化计算每个顶点的 h 值，这需要扫描所有顶点和边，需要 $O(n+m)$ 时间，其中 $n = |V|, m = |E|$。每轮扫描 h 数组寻找不满意顶点，再更新它及其邻居，各至多需要 $O(n)$ 时间。如果共翻转 T 次，总时间为 $O(m + n(T+1))$。

显然，总时间复杂度的关键在于翻转次数 T。如果算法能在有限步内停止（即 $T < \infty$），所有顶点都满意，得到的自然就是稳定构型。但它一定会停止吗？翻转一个顶点虽然让它自己满意，却可能让邻居不满意，而邻居的翻转又可能反过来影响它。要排除这种反复影响造成的循环，需要观察整个构型在每一步发生了什么变化。

实际上，当翻转 u 时，只有与它相连的边改变好坏。原来坏边的权重绝对值之和是 b_u，翻转后都变成好边；原来好边的权重绝对值之和是 g_u，翻转后都变成坏边。因此，好边权重之和增加了 $b_u - g_u$。

而算法只翻转不满意的点，恰好保证这个增量为正。这就提示我们选择如下函数：

$$\Phi(s) = \sum_{e \text{ 为好边}} |w_e|.$$

令 $W = \sum_{e \in E} |w_e|$，则对任何构型都有 $0 \leqslant \Phi(s) \leqslant W$。每次翻转不满意顶点 u 后得到 s'，则 $\Phi(s') - \Phi(s) = b_u - g_u = h[u] > 0$，所以 Φ 严格增加。因为每次翻转都使 Φ 严格增加，所以已经出现过的构型不会再次出现。n 个顶点各有两种状态，共有 2^n 个构型，因此最多翻转 $2^n - 1$ 次。进一步地，如果边权都是整数，则 Φ 也是整数，每次增加的正数至少为 1，所以翻转次数还满足 $T \leqslant W - \Phi(s_0) \leqslant W$，其中 s_0 是初始构型。我们将上述讨论总结为下面的命题。

> **定理 12.2.** 对任意实数边权，严格单点翻转至多进行 $2^n - 1$ 次后停止。若边权都是整数，则还可得到 $T \leqslant W - \Phi(s_0) \leqslant W$。

我们称上述方法为**势函数法（potential function method）**，这是分析局部搜索算法的常用工具。势函数把许多顶点之间的相互影响，转化为一个始终增加的数值，从而同一状态不会重复出现，然后根据状态的有限性来保证算法的终止性。当然我们也可以改用 $E(s) = W - \Phi(s)$，让每次改进都表现为能量下降。

进一步地，我们还可以把稳定构型与势函数的局部最优联系起来。

> **定理 12.3.** 构型 s 是稳定构型，当且仅当 $\Phi(s)$ 是在单点翻转邻域下的局部最大值。

证明. 若存在不满意顶点 u，翻转它使 Φ 增加 $h[u] > 0$，所以当前构型不是局部最大值。反之，若所有顶点都满意，翻转任意一点的增量都是 $h[u] \leqslant 0$，没有更好的邻居，因此当前构型是局部最大值。 □

根据上面两个定理，算法一定终止，并且只在稳定构型处终止，因此每个这样的网络都存在稳定构型，而且寻找局部最优就能得到我们想要的答案。然而算法的终止性并不意味着它能在多项式时间内完成。定理 12.2 在边权为实数时给出了指数级的上界，边权为整数时的上界 $O(W)$ 也是伪多项式界。事实上，能否换一种方法更快找到稳定构型，是补充材料（第12.6.3节）中介绍的 PLS 复杂度类所研究的问题之一。

12.5 最大割问题

给定一个无向图，我们希望把它的顶点分成 A, B 两组，让两组之间的边尽可能多。如果每条边还带有权重，就希望这些边的权重之和尽可能大，这就是**最大割问题（maximum cut problem）**。两个端点分别属于 A 和 B 的边称为割边。以下设图 $G = (V, E)$ 没有自环，边权为正数，并记当前划分的割边权重之和为

$$w(A, B) = \sum_{\{u,v\} \in E,\, u \in A,\, v \in B} w_{uv}.$$

最大割也是一个经典的 NP 困难问题，因此下面尝试用局部搜索寻找近似解。

12.5.1 局部最优的近似保证

一个自然的想法是先任意分组，再逐个检查顶点。如果把某个顶点移到另一组能够增加割边的总权重，就移动它，直到所有顶点都无法这样改进。事实上，这就是上一节的状态翻转算法：将 A 中

的点赋状态 -1，B 中的点赋状态 $+1$，对于正权边，两端状态不同就意味着它是好边，也恰好意味着它是割边。因此，好边权重之和 Φ 就等于这里的 $w(A,B)$，翻转一个点就相当于让它换组。

上一节已经证明，算法最终会停在一个局部最优解。不过，这次我们还希望知道它与全局最优相差多少。下面的定理表明，算法得到的割边总权重至少是最优值的一半。

定理 12.4. 设 (A,B) 为算法当前步骤的划分，$w(A,B)$ 为此时的割边权重之和。记全部边的权重之和为 $W=\sum_{e\in E} w_e$，最优割的权重为 OPT。如果将任意一个顶点移到另一组都不能增加割边的总权重，那么

$$OPT \leqslant W \leqslant 2w(A,B).$$

证明. 取一个顶点 $u\in A$。如果将它移到 B，原来连向 A 的边就会变成割边，原来连向 B 的边则不再是割边。由于算法已经停止，这次移动增加的权重不可能为正，所以

$$\sum_{v\in A:\{u,v\}\in E} w_{uv} \leqslant \sum_{v\in B:\{u,v\}\in E} w_{uv}.$$

将 A 内部的边权之和记为 I_A。对每个 $u\in A$ 都能得到上述不等式，将它们相加，便得到 $2I_A \leqslant w(A,B)$。同样地，对 B 中的顶点求和，有 $2I_B \leqslant w(A,B)$，其中 I_B 是 B 内部的边权之和。

全部边可以分为 A 内部的边、B 内部的边和割边三部分，所以

$$W = I_A + I_B + w(A,B) \leqslant \frac{w(A,B)}{2} + \frac{w(A,B)}{2} + w(A,B) = 2w(A,B).$$

任何割都只包含图中的一部分边，边权又都非负，因此 $OPT \leqslant W \leqslant 2w(A,B)$，结论得证。 □

在这个证明中，每个顶点都不愿换组，最终限制了整个划分与最优解的差距。习题 12.6 会用类似的思路分析同学选择助教的问题，那里每位同学也都不愿单独改变自己的选择。

不过，要证明这是 2-近似算法，还需要保证运行时间是多项式。上一节证明，边权为整数时至多翻转 W 次，其中 W 是所有边的权重之和。如果边权都为 1，那么 $W=m$，翻转次数就不超过边数，每次翻转也能在多项式时间内完成。但一般边权可能很大，因此 W 次翻转仍可能远多于输入长度的任何固定次幂。要限制运行时间，还需要进一步减少翻转次数。

12.5.2 控制改进次数

为了控制翻转次数，我们可以提高继续搜索的要求，只接受足够大的改进，如果剩下的改进都太小，就提前结束。这样可能放弃一部分改进机会，但只要能估计损失，就仍然能得到近似保证。具体地，设 $n=|V|$，给定 $0<\varepsilon\leqslant 1$。当前割边权重和为 $w(A,B)$ 时，只有某个顶点翻转后能使割权增加至少 $\varepsilon w(A,B)/n$，才执行这次翻转。如此一来，每次翻转后的割权至少为原来的 $1+\varepsilon/n$ 倍。

定理 12.5. 设 (A,B) 是上述算法停止时得到的划分，最优割的权重为 OPT，则

$$w(A,B) \geqslant \frac{OPT}{2+\varepsilon}.$$

若边权为正整数，从任意割权为正的初态出发，算法会在 $O((n/\varepsilon)\log W)$ 次翻转后停止，其中 W 是所有边的权重之和。

证明. 近似比的证明只需在前一定理的证明中加入提前停止所允许的误差。算法停止时,将任意 $u \in A$ 移到 B 所增加的割权都小于 $\varepsilon w(A,B)/n$，因此

$$\sum_{v\in A:\{u,v\}\in E} w_{uv} \leqslant \sum_{v\in B:\{u,v\}\in E} w_{uv} + \frac{\varepsilon w(A,B)}{n}.$$

仍用 I_A, I_B 分别表示 A, B 内部的边权之和。对所有 $u \in A$ 求和，有

$$2I_A \leqslant w(A,B) + |A|\frac{\varepsilon w(A,B)}{n} \leqslant (1+\varepsilon)w(A,B).$$

同样地，对 B 中的顶点求和，得到 $2I_B \leqslant (1+\varepsilon)w(A,B)$。与前一定理一样，将两组内部的边和割边加起来，便有

$$OPT \leqslant W = I_A + I_B + w(A,B) \leqslant (2+\varepsilon)w(A,B).$$

这就证明了所需的近似比。

再看翻转次数。每次翻转都将割权至少乘以 $1+\varepsilon/n$，因此连续翻转 $\lceil n/\varepsilon \rceil$ 次后，割权至少翻倍。这是因为对正整数 k，二项式展开给出 $(1+x)^k \geqslant 1+kx$。由于边权为正整数，初始割权至少为 1。割权最多为 W，因此算法会在 $O((n/\varepsilon)\log W)$ 次翻转后停止。 □

还可以通过选择初始划分，得到与 W 无关的翻转次数上界。我们可以先选一个顶点单独作为一组，其余顶点放在另一组。这时割权就是与该点相连的边权之和，我们选择这个和最大的顶点。对所有顶点求这些和，每条边都被计了两次，所以最大的一个至少为平均值 $2W/n$。记这样得到的初始划分为 (A_0, B_0)，则 $w(A_0, B_0) \geqslant \frac{2W}{n}$。采用前面的初始化方法，初始割权至少为 $2W/n$，最终又不超过 W，所以最多只能增长到初始值的 $n/2$ 倍。完整的翻倍过程至多发生 $\lfloor \log_2(n/2) \rfloor$ 次，故总次数为 $O((n/\varepsilon)\log n)$。事实上这一结论适用于任意正边权，不需要整数边权的假设。

12.5.3 邻域的选择

前面一直把只翻转一个点得到的划分作为邻居。但即使一个点单独换组不能改善结果，几个点一起换组也有可能改善。为了找到这样的机会，可以允许同时翻转至多 k 个顶点，得到 k-flip 邻域。

增大 k 后，原来能够选择的邻居仍然保留，还会增加允许翻转更多顶点的新邻居。因此，如果一个解在 k'-flip 邻域中已经是局部最优，那么对于任何 $k < k'$，它也一定是 k-flip 局部最优。反过来却未必成立，这正是扩大邻域有可能继续改善结果的原因。

扩大邻域还会增加寻找改进所需的时间。允许翻转至多 k 个点，就要考虑

$$\sum_{i=1}^{\min\{k,n\}} \binom{n}{i}$$

个点集，对于固定的 k，数量为 $O(n^k)$。即使每个点集的增量都能很快算出，k 稍大时，全部检查一遍也很费时间。极端地，如果把所有划分都作为邻居，寻找最好的邻居就变成了直接求全局最优。因此，我们希望能检查一些有希望的多点改动，又不必枚举所有组合。

Kernighan 和 Lin（1970）提出的 K-L 启发式方法提供了一种权衡，通过如下算法得到 (A, B) 的候选邻居。需要注意的是，原论文研究的是两组人数相等的最小割划分，因此每次交换两组各一个点，此处我们将其改编用于最大割，每次翻转一个点。

1. 第一步，选取一个点进行翻转，要求选取的这个点是让翻转后结果的割边权重和最大的那个点（即使最大的权重可能会比现有权重低），标记这个被翻转的点，令翻转后的解为 (A_1, B_1)。
2. 之后第 i（$i > 1$）步时，有前一步得到的结果 (A_{i-1}, B_{i-1})，以及 $i-1$ 个已经被标记的点。然后选择一个没有被标记的点进行翻转，要求选取的这个点是让翻转后结果的割边权重和最大的那个点，标记这个被翻转的点，令翻转后的解为 (A_i, B_i)。
3. n 步之后所有点都被标记了，也就是说所有的点都恰好被翻转了一次，因此有 $(A_n, B_n) = (B, A)$，这本质上与 (A, B) 并无区别。因此上述过程得到了 $n-1$ 个候选邻居，即

$$(A_1, B_1), (A_2, B_2), \ldots, (A_{n-1}, B_{n-1}).$$

   相对于这样定义的邻域，(A, B) 是局部最优解当且仅当对于任意的 $1 \leqslant i \leqslant n-1$，都有 $w(A, B) \geqslant w(A_i, B_i)$。

如果这些候选邻居中有比 (A, B) 更好的划分，就选取割权最大的一个作为下一轮的起点，重复上述过程，否则搜索结束。

上述算法每轮共翻转 n 个顶点，每次翻转需要扫描尚未选过的顶点寻找最大增量，故总共需要 $O(n^2)$ 时间。每次翻转一个顶点，都要遍历它的邻接表，更新相邻顶点的翻转增量。一轮中每个顶点只翻转一次，因此这部分总共需要 $O(m)$ 时间。因此每轮时间为 $O(n^2 + m)$。注意 K-L 方法保留了前面证明的算法终止性以及解的质量保证，因为其考虑了所有单点翻转的邻居。

12.6 补充材料

12.6.1 全局优化的复杂度界

假如我们想找到一片地形的最低点，但只能查询当前选定位置的高度，那么少量查询未必能发现藏在别处的深谷。把这个想法写成数学问题，就能说明为什么全局优化有时需要很多次查询，从而说明全局优化的困难。

设要最小化的函数为 f，每个变量都在 $[0, 1]$ 中，因此可行域是 n 维盒子 $\mathbb{B}^n = [0, 1]^n$。我们不知道 f 的表达式，只能给出一个点 x，取得函数值 $f(x)$。我们把提供函数值的过程称为 **Oracle**。为了使未查询点的值也受到一定限制，假设已知常数 $L \geqslant 0$，且 $|f(x) - f(y)| \leqslant L\|x - y\|_\infty$，其中 $\|x - y\|_\infty = \max_{1 \leqslant i \leqslant n} |x_i - y_i|$。这就是关于 ℓ_∞ 距离的 **Lipschitz 连续条件**。它表示，只要每个坐标的改变量都不超过 d，函数值的变化就不超过 Ld。

给定误差要求 $\varepsilon > 0$，我们希望找到 $\bar{x}$，使 $f(\bar{x}) - f^* \leqslant \varepsilon$，$f^* = \min_{x \in \mathbb{B}^n} f(x)$。Lipschitz 条件保证函数连续，所以它在这个闭且有界的盒子上能够取到最小值。下面只统计查询函数值的次数，称为查询复杂度。

最直接的算法是选取足够密的网格，把各个网格点的函数值都算出来，再返回其中最小的一个。以二维为例，把正方形的每条边均分为 p 份，就有 $(p+1)^2$ 个网格点；在 n 维中，则有 $(p+1)^n$ 个。网格越密，真正的最小值点到最近网格点的距离越小，再利用 Lipschitz 条件就能控制误差。这称为**均匀网格法**。当 $L = 0$ 时，任意两点的函数值都相同，直接返回一个点即可。对于 $L > 0$ 的情况，下面证明算法中的 p 与精度的关系。

> **定理 12.6.** 均匀网格法返回的点满足 $f(\bar{x}) - f^* \leqslant L/(2p)$。

证明. 设 x_* 是一个最小值点。将它的每个坐标舍入到最近的网格坐标，得到网格点 $\tilde{x}$。相邻网格坐标相差 $1/p$，所以舍入误差至多为一半，即 $\|\tilde{x} - x_*\|_\infty \leqslant \frac{1}{2p}$。算法返回的 $\bar{x}$ 是网格点中函数值最小的一个，当然不比 $\tilde{x}$ 更差。因此

$$f(\bar{x}) - f(x_*) \leqslant f(\tilde{x}) - f(x_*) \leqslant L\|\tilde{x} - x_*\|_\infty \leqslant \frac{L}{2p}.$$

□

因此，令 $p = \lceil L/(2\varepsilon) \rceil$，查询次数为 $(p+1)^n$，就能保证误差不超过 ε。

上面的结论给出了得到指定精度所需的网格密度，从而得到了查询次数的上界。然而我们希望得到查询次数的下界，从而说明全局优化问题的困难。显然，均匀网格法无论已经得到什么结果，都会查完预定的全部点。如果根据已有结果决定下一次查询哪里，能否少查很多点呢？下面证明，即使允许这样选择，最坏情况下仍然需要大量查询。证明的办法是先对每次查询都回答 0。如果算法查得太少，就在它没有查到、也没有作为答案输出的地方放一个足够深的山谷。只要构造出的函数满足 Lipschitz 条件，且在所有已查询点上仍为 0，算法就无法察觉这个山谷。这种根据算法行为构造输入的证明方法称为**对抗论证**。

> **定理 12.7.** 设 q 是满足 $1 \leqslant q < L/(2\varepsilon)$ 的整数。任何保证对上述所有函数都返回误差不超过 ε 的确定性算法，最坏情况下至少需要 $q^n - 1$ 次查询。

证明. 把盒子每条边均分为 q 份，得到 q^n 个小盒子。对算法的每次查询都返回 0，假设它在 $N < q^n - 1$ 次查询后停止，输出 $\bar{x}$。查询点加上输出点一共至多 $N + 1 < q^n$ 个，而这些小盒子的内部互不相交，所以至少有一个小盒子的内部没有这些点。我们要把山谷放在这个盒子里，并使盒子边界及外部的函数值保持为 0。

设盒子中心为 z，定义

$$f_z(x) = \min\left\{0, L\|x - z\|_\infty - \frac{L}{2q}\right\}.$$

小盒子的边长为 $1/q$，所以边界到中心的 ℓ_∞ 距离为 $1/(2q)$。上式在边界及外部为 0，在内部逐渐降低，到中心时取到最小值 $-L/(2q)$。由于距离函数满足 $\big|\|x - z\|_\infty - \|y - z\|_\infty\big| \leqslant \|x - y\|_\infty$，乘上 L，再把正值改为 0，都保持所要求的 Lipschitz 界，因此 f_z 是满足要求的函数。

由于所有查询点和输出点都在这个盒子的内部之外，故它们的函数值均为 0。因此，算法的误差为 $L/(2q) > \varepsilon$，与精确度保证矛盾。 □

令 $p = \lceil L/(2\varepsilon) \rceil$。当 $\varepsilon < L/2$ 时，可以取 $q = p - 1$，所以查询次数的上、下界分别为 $(p+1)^n$ 和 $(p-1)^n - 1$。固定维数 n，令精度要求越来越高，两者都是 $\Theta((L/\varepsilon)^n)$。特别是，维数出现在指数上，即使每个坐标只需检查少量位置，组合起来也可能有很多点。

这里的困难来自我们掌握的信息太少：只知道函数变化不太快，仍无法排除某个未访问区域藏着更低的值。如果还能利用函数的表达式或凸性，就可能设计更快的算法，如梯度下降法。

12.6.2 固定温度下的收敛证明

以下沿用定理 12.1 的条件，固定温度，并将 π_θ 简记为 π。正文已经证明 π 是平稳分布，下面还需要证明：从任意初态出发都能趋近 π，以及长期访问一个状态的次数比例也趋于 π 给出的概率。

首先证明收敛性。设共有 K 个状态。$K=1$ 时是平凡的，以下设 $K\geqslant 2$，并记一步转移矩阵为 P。由提议的连通性和接受概率为正，任意两个状态之间都有一条不重复经过状态的转移路径，长度至多为 $K-1$。算法每轮还以至少 $1/2$ 的概率原地停留，所以可以用停留补足步数，使这条路径恰好有 $r=K-1$ 步。每一步的概率都为正，因此 P^r 的每个元素都为正。这意味着，不论从哪个状态出发，经过 r 步后，都有机会到达所有状态。取 $\beta=\min_{i,j}(P^r)_{ij}>0$，$\eta=K\beta/2$。因为每行之和为 1，所以 $K\beta\leqslant 1$，从而 $0<\eta<1$。令 U 的每行都是均匀分布，则可以写成

$$P^r=\eta U+(1-\eta)Q,\qquad Q=\frac{P^r-\eta U}{1-\eta}.$$

ηU 的每个元素为 $\beta/2$，所以相减后仍非负。再除以 $1-\eta$，每行之和恢复为 1。因此 Q 也是转移矩阵。上式表明，经过这 r 步后得到的状态分布，等价于以概率 η 均匀选择一个状态，以剩余概率按照 Q 从出发状态转移。均匀选择时，每个状态被选中的概率都是 $1/K$，与出发状态无关。

用概率行向量 μ,ν 表示两种初始分布。因为 $\mu U=\nu U$，均匀分布部分不会留下差别。Q 对应的部分也不会扩大两种分布的 ℓ_1 距离：对任意行向量 a，

$$\sum_j\left|\sum_i a_iQ_{ij}\right|\leqslant\sum_{i,j}|a_i|Q_{ij}=\sum_i|a_i|.$$

因此，每经过 r 步，距离至少按因子 $1-\eta$ 缩小。令 $\nu=\pi$，由于任意两分布的初始距离至多为 2，利用平稳性有

$$\|\mu P^t-\pi\|_1\leqslant 2(1-\eta)^{\lfloor t/r\rfloor}\longrightarrow 0.$$

由此也可以看出平稳分布是唯一的：若 μ 也是平稳分布，则 $\mu P^t=\mu$，所以前面的收敛结论说明 $\|\mu-\pi\|_1=0$，即 $\mu=\pi$。

接下来考虑访问次数的比例。固定一个状态 R，令 Y_i 表示第 i 步是否位于 R，在 R 时为 1，否则为 0，则前 t 步的访问次数就是 $N_t=\sum_{i=1}^{t}Y_i$。由分布收敛，$\mathbb{E}[Y_i]\to\pi(R)$，这些期望的平均值也趋于同一极限，所以

$$\mathbb{E}\left[\frac{N_t}{t}\right]\longrightarrow\pi(R).$$

还需证明实际频率与这个期望之间的差也趋于零。

要证明实际访问比例接近它的期望，可以先估计访问次数 N_t 的方差。这里的 Y_i 并不独立，因为当前状态会影响后面的状态。不过，前面已经证明，无论从哪里出发，运行足够多步后都接近同一个分布 π。因此，两次访问相隔越久，前一次状态对后一次的影响就越小。

记第 i 步的状态为 X_i。由于 Y_j 表示第 j 步是否位于 R，$\mathbb{E}[Y_j\mid X_i]$ 就是在已知 X_i 后，第 j 步位于 R 的概率。从 X_i 再运行 $j-i$ 步，利用前面的收敛界，有

$$|\mathbb{E}[Y_j\mid X_i]-\pi(R)|\leqslant 2(1-\eta)^{\lfloor(j-i)/r\rfloor}.$$

不指定 X_i 时，只需对第 i 步可能出现的状态按其概率取平均，因此 $\mathbb{E}[Y_j]$ 与 $\pi(R)$ 的差也满足同一上界。由三角不等式，

$$|\mathbb{E}[Y_j\mid X_i]-\mathbb{E}[Y_j]|\leqslant 4(1-\eta)^{\lfloor(j-i)/r\rfloor}.$$

Y_i 由 X_i 决定，所以先给定 X_i 再取期望，可以将协方差写成

$$\operatorname{Cov}(Y_i,Y_j)=\mathbb{E}[Y_iY_j]-\mathbb{E}[Y_i]\mathbb{E}[Y_j]=\mathbb{E}\big[Y_i\big(\mathbb{E}[Y_j\mid X_i]-\mathbb{E}[Y_j]\big)\big].$$

又因为 $0 \leqslant Y_i \leqslant 1$，协方差的绝对值也不超过上面的界。固定 i，将后面的时刻按相隔的步数分组，每 r 步一组，便得到一个等比数列的和：

$$\sum_{j=i+1}^{t} |\operatorname{Cov}(Y_i, Y_j)| \leqslant 4r \sum_{k=0}^{\infty} (1-\eta)^k = \frac{4r}{\eta}.$$

由于每个 Y_i 的方差至多为 1，于是

$$\operatorname{Var}(N_t) = \sum_{i=1}^{t} \operatorname{Var}(Y_i) + 2 \sum_{1 \leqslant i < j \leqslant t} \operatorname{Cov}(Y_i, Y_j) \leqslant \left(1 + \frac{8r}{\eta}\right) t = Ct,$$

其中 $C = 1 + 8r/\eta$ 与 t 无关。由 Chebyshev 不等式，对任意 $a > 0$，

$$\Pr\left(\left| \frac{N_t}{t} - \mathbb{E}\frac{N_t}{t} \right| > a \right) \leqslant \frac{C}{a^2 t}.$$

我们还要说明，这样的偏差以概率 1 会一直小于 a。直接将上述概率对全部时刻求和会遇到发散的调和级数，因此先只考虑 $t = j^2$。在 $J^2, (J+1)^2, \ldots$ 这些时刻中，至少有一次偏差大于 a 的概率，至多为

$$\sum_{j=J}^{\infty} \frac{C}{a^2 j^2} \longrightarrow 0 \qquad (J \to \infty).$$

结合前面得到的期望收敛，就有 $N_{j^2}/j^2 \to \pi(R)$。考虑夹在两个平方时刻之间的任意时刻 $j^2 \leqslant t < (j+1)^2$。从 j^2 到 t 最多增加 $2j+1$ 步，而每步至多访问 R 一次，所以 $0 \leqslant N_t - N_{j^2} \leqslant t - j^2$，故有

$$\left| \frac{N_t}{t} - \frac{N_{j^2}}{j^2} \right| \leqslant \frac{N_t - N_{j^2}}{t} + N_{j^2}\left(\frac{1}{j^2} - \frac{1}{t}\right) \leqslant \frac{2(t - j^2)}{t} \leqslant \frac{2(2j+1)}{j^2} \longrightarrow 0.$$

因此，平方时刻之间的访问比例也趋于 $\pi(R)$，命题得证。

12.6.3 局部搜索的复杂度

前面已经多次遇到同一个问题：一次改进可以很快完成，但反复改进直到局部最优，可能要走很多步。这是否说明寻找局部最优本身就很难呢？还不能这样下结论，因为另一种算法可能换一条更短的路径，甚至直接构造出答案。我们真正想问的是，能否在多项式时间内找到任意一个局部最优解。

Johnson、Papadimitriou 和 Yannakakis 为研究这个问题，引入了复杂度类 PLS（polynomial local search）。相关工作发表于 FOCS 1985 和 1988 年的正式论文 *How Easy Is Local Search?*。这里的 polynomial 指每次搜索所需的基本操作能够高效完成。

具体而言，给定实例 I 后，可行解集合有限，每个解都能用关于输入长度 $|I|$ 的多项式个位表示，目标值为多项式位数的非负整数。此外，还应有三个多项式时间过程：

1. 产生一个初始可行解；
2. 检查一个给定编码是否表示可行解，并计算其目标值；
3. 对一个可行解，找到一个严格更好的邻居，或者确认不存在这样的邻居。

任务是找到一个无法继续严格改进的可行解。满足这些条件的局部最优搜索问题组成 PLS。整数边权下的 Hopfield 稳定构型搜索就满足这些条件。可以用全 $+1$ 的状态初始化，以 $W - \Phi$ 为目标，再检查每个点的 $h[u]$ 来决定改进或停止。

与 NP 完全性类似，我们也可以用归约比较不同局部最优搜索问题的难度。不过，之前判定问题的归约只需保证转换前后的答案同为“是”或同为“否”，这里要找出具体的局部最优解，因此还要求能将目标实例的任意局部最优解，在多项式时间内转换成原实例的局部最优解，所以归约需要做两次转换：先把原实例 I 转成目标问题的实例，再把目标实例的任意合法答案转回 I 的合法答案。两次转换都必须在多项式时间内完成。如果一个问题属于 PLS，且所有 PLS 问题都能这样归约到它，就称它为 PLS 完全问题。

Circuit-FLIP 就是一个 PLS 完全问题。给定一个布尔电路，每组输入都会产生一串二进制输出，我们把这串输出表示的非负整数作为目标值，把只改变一个输入位得到的输入作为邻居。问题要求找到一组输入，使改变其中任何一位都不能让输出表示的整数变大。这个问题满足前面的三个要求，可以从全零输入开始，通过计算电路得到目标值，再依次尝试改变每个输入位，检查是否存在更好的邻居。这些操作都能在多项式时间内完成，但要找到一个局部最优输入，可能需要反复改进多次。

前面讨论的稳定构型也有这样的困难。Schäffer 与 Yannakakis（1991）证明，给定整数边权的无向连接网络，寻找一个稳定构型是 PLS 完全问题。因此，假如我们找到一种能对所有这类网络在多项式时间内求出稳定构型的算法，就可以先将任意 PLS 问题的实例归约成网络，再用这个算法求解，最后把稳定构型转换回原问题的局部最优解。这样一来，所有 PLS 问题都能在多项式时间内解决。然而与 NP 完全问题一样，PLS 完全问题目前也没有已知的多项式时间算法。

12.6.4 必有解的搜索问题

前面介绍的局部最优搜索问题都有解。只要可行解集合有限且非空，其中就有一个目标值最好的解，而它当然也是局部最优解。因此，我们关心的是怎样找到局部最优解，而无须先判断它是否存在。复杂度理论把这类问题放在一起研究，下面介绍它们所属的复杂度类。

以布尔公式为例，判定问题关注公式能否满足，搜索问题则要求给出一组满足赋值。对搜索问题，通常要求答案长度不超过输入长度的某个多项式，并且给定一个候选答案后，能够在多项式时间内检查它是否正确。满足这两个要求的搜索问题组成 FNP（function NP）。

然而公式可能不可满足，此时搜索问题没有答案。如果进一步要求每个输入都至少有一个答案，就得到 TFNP（total function NP）。PLS 属于其中，因为局部最优解一定存在，而且定义中的第三个过程能够检查给定解是否已经局部最优。因此有 PLS $\subseteq$ TFNP。这些问题虽然都有解，却未必能很快找到。如果存在一个确定性多项式时间算法，能为每个输入找出一个合法答案，我们就简称这个搜索问题属于 FP。严格来说，FP 中的对象是函数，这里指的就是算法为每个输入选出一个答案所定义的函数。

还有一些问题可以用图的性质保证答案存在。假设一个有限有向图中，每个顶点至多有一条入边和一条出边，并且已经找到一个没有入边、但有出边的顶点。从这里沿着出边一直走，最终一定会到达一个没有出边的顶点。否则，有限个顶点中必然有顶点被重复访问，也就形成了环。但路径第一次进入环时，入口同时有来自环外和环内的两条入边，与条件矛盾。如果整张图已经列出，沿着路径寻找终点，访问的顶点数不会超过图的顶点总数。但有时简短的输入也可能描述一张很大的图，沿路径逐个访问顶点就未必能在关于输入长度的多项式时间内完成。**End-of-Line** 问题研究的就是这种情况下如何寻找另一个端点。

具体地，每个顶点用一个 b 位二进制串表示，输入给出两个布尔电路 P 和 S。将表示顶点 u 的二进制串输入电路后，P 输出一个顶点 $P(u)$，表示希望从哪个顶点连到 u，S 输出一个顶点 $S(u)$，表示希望从 u 连到哪个顶点。两个电路的结果必须一致才真正连边，即对于不同的顶点 u, v，只有同时

满足 $S(u) = v$ 和 $P(v) = u$，才有边 $u \to v$。因此，每个顶点仍然至多有一条入边和一条出边。问题还给出一个没有入边、但有出边的源点，要求找到另一个只有入边或只有出边的顶点。前面的论证保证答案存在，而利用这两个电路，也能在关于输入长度的多项式时间内检查一个顶点是否为所求端点。困难在于如何找到它。b 位串可以表示 2^b 个顶点，因此，即使电路很小，从已知源点沿路径寻找终点也可能需要经过指数多个顶点。

能够在多项式时间内归约到 End-of-Line 的搜索问题组成 PPAD（polynomial parity arguments on directed graphs）。不难看出，PPAD 也属于 TFNP。

介绍了 PLS 和 PPAD 之后，我们也可以问，能否像处理 SAT 那样，用一个必有解的搜索问题来解决所有 FNP 问题？下面说明，如果可以，就会推出 NP = co-NP。

假设有一个属于 TFNP 的问题，所有 FNP 问题都能归约到它。特别地，寻找公式满足赋值的问题也能这样归约。给定公式 F，先在多项式时间内构造目标实例 I_F。按照归约的要求，只要 F 可满足，I_F 的任意合法答案都必须能转换成 F 的满足赋值。而由于目标问题属于 TFNP，即使 F 不可满足，I_F 也仍然有答案。这就给了我们一种验证 F 不可满足的方法。把 I_F 的一个合法答案作为证据，先检查这个答案是否正确，再将它转换成赋值，检查该赋值是否满足 F。若答案正确，但转换得到的赋值不满足 F，就能断定 F 不可满足。反过来，若 F 确实不可满足，I_F 仍有合法答案，而转换后必然无法满足 F。上述归约表明 I_F 的答案是一份多项式长度的证据，能多项式时间验证 F 是否不可满足，因此 UNSAT 属于 NP，从而 NP = co-NP。

12.6.5 纳什均衡与局部搜索

事实上，博弈论中纳什均衡的概念与局部搜索有着密切联系。前面的 Hopfield 网络可以看作一个博弈。把每个顶点当作一位参与者，让它选择自己的状态，并把与它相连的好边的权重绝对值之和作为收益。如果一个顶点不满意，它就能通过翻转提高自己的收益。如果所有顶点都满意，就没有人愿意单独改变状态。这正是纳什均衡所描述的情形。

> **定义 12.8** (纯策略与混合策略纳什均衡)**.** 在一个博弈中，每位参与者都有可供选择的行动，所有人的行动共同决定各自的收益。直接选定一个行动称为采用**纯策略**。如果其他人的行动保持不变时，任何人都不能通过单独改变自己的行动获得更高收益，那么这组行动就构成**纯策略纳什均衡（pure-strategy Nash equilibrium）**。
> 参与者也可以为各个行动指定概率，这称为采用**混合策略**。各人独立选择行动，并用期望收益比较策略。如果其他人的概率分布保持不变时，任何人都不能通过改变自己的概率分布提高期望收益，就称这组策略为**混合策略纳什均衡（mixed-strategy Nash equilibrium）**。以概率 1 选定某个行动也是混合策略，因此纯策略均衡是混合策略均衡的特殊情形。

具体到 Hopfield 网络，顶点 u 的收益原来是 g_u，翻转后变为 b_u，所以它能够通过翻转获益当且仅当 $b_u > g_u$。这与不满意顶点的条件相同，因而稳定构型恰好就是这个博弈的纯策略纳什均衡。这里还能看出前面势函数的作用。顶点 u 翻转后，个人收益增加 $b_u - g_u$，整个网络的势函数 Φ 也恰好增加这么多。一般地，如果无论哪位参与者单独改变行动，其收益的变化都等于同一个函数的变化，就称这个函数为**精确势函数**，相应的博弈称为**精确势博弈**。当参与者和行动数有限时，只要每次都让一位参与者严格提高收益，势函数就严格增加，已经出现过的行动组合便不会重复。于是这一过程一定会停止，停下时就是纯策略纳什均衡。

最大割也有同样的解释。让每个顶点选择自己属于哪一组，并以与自己相连的割边权重之和作

为收益，那么单独换组带来的个人收益变化，就等于总割权的变化。因此，总割权是精确势函数，单点翻转的局部最优划分就是纯策略纳什均衡。

一般博弈未必有纯策略均衡。例如，两个人同时选择硬币的正面或反面，甲在两人选择相同时获胜，乙在不同时获胜，胜者收益为 1，败者为 -1。无论两人选定什么，败者都能通过改变自己的选择获胜，所以没有纯策略均衡。但如果双方都以 $1/2$ 的概率选择正反面，那么只要一方保持这个分布，另一方无论怎样选择，胜率都为 $1/2$。双方都无法单独提高期望收益，这就得到了混合策略均衡。

Nash 在 1950 年证明，每个参与者和纯策略数都有限的博弈，都至少有一个混合策略纳什均衡。因此，寻找这样的均衡也是一个必有答案的搜索任务。下面考虑双人博弈，用两个矩阵 A, B 表示收益。当第一人选择第 i 个行动、第二人选择第 j 个行动时，两人的收益分别为 A_{ij} 和 B_{ij}，我们要求输出双方在一个均衡中的策略概率。陈汐、邓小铁在 FOCS 2006 证明，求解双人矩阵博弈的精确纳什均衡是 PPAD 完全问题。陈汐、邓小铁、滕尚华于 2009 年发表的论文还给出了近似均衡计算的困难性结果。具体而言，先将收益限制在 $[0,1]$，允许每位参与者通过单独改变策略至多增加 ε 的期望收益，这样得到的策略组合称为 ε-纳什均衡。除非全部 PPAD 问题都能在多项式时间内解决，否则不存在运行时间同时关于输入长度和 $1/\varepsilon$ 为多项式的通用求解算法。

12.6.6 连续局部搜索

前面的局部搜索只有有限个可行解，只要每次严格改进，就一定会停止。换成实数变量后，这个理由就不再适用。例如，为了减小 $f(x) = x$，从 $x = 1$ 开始，每次将 x 减半，虽然每步都有改进，却永远到不了最优点 0。如果允许误差，就可以在下一步带来的改进已经很小时停止。

形式化地，我们同时给出目标函数 p 和决定下一步怎样移动的函数 g。在当前点 x，算法尝试移动到 $g(x)$，比较的就是 $p(x)$ 与 $p(g(x))$。连续局部搜索的一个标准模型在三维单位立方体上考虑这两个函数，要求 $p : [0,1]^3 \to [0,1]$，$g : [0,1]^3 \to [0,1]^3$。给定有理数 $\delta > 0$ 后，我们要找到一个用有限位数表示的有理点 x，使 $p(g(x)) \geqslant p(x) - \delta$。也就是说，按照给定的移动方法再走一步，目标值至多下降 δ。这里的 p 和 g 由算术电路给出，电路采用受限制的运算，以保证对有理输入可以在多项式时间内计算函数值。输入还给出一个正数 L，要求两个函数都满足 Lipschitz 条件，即

$$|p(x) - p(y)| \leqslant L\|x - y\|_2, \qquad \|g(x) - g(y)\|_2 \leqslant L\|x - y\|_2.$$

例如，令 $p(x) = x_1$，$g(x) = (x_1/2, x_2, x_3)$，就得到了前面不断减半的过程。这时停止条件为 $x_1/2 \geqslant x_1 - \delta$，也就是 $x_1 \leqslant 2\delta$。我们无须等到 x_1 精确等于零，只需找到满足这个不等式的点。

一般地，目标值始终在 $[0,1]$ 内，如果每次都下降超过 δ，这样的改进不可能进行超过 $1/\delta$ 次。然而，输入中的 δ 可能很小，例如 $\delta = 2^{-b}$ 只需 $O(b)$ 位表示，$1/\delta$ 却等于 2^b。因此，这个次数上界还不能保证算法在关于输入长度的多项式时间内完成。

上面要求 p 和 g 都满足由 L 给出的 Lipschitz 条件，但 L 也是输入的一部分，实际的函数可能不满足这个条件。对于这样的输入，问题允许算法找出一对违反条件的点作为答案，也就是给出 x, y，使得 $|p(x) - p(y)| > L\|x - y\|_2$ 或 $\|g(x) - g(y)\|_2 > L\|x - y\|_2$。如果输入的 L 满足要求，就不会有这样的反例，算法仍须找到前面要求的近似停止点。我们称能够在多项式时间内归约到这一问题的搜索问题组成 CLS（continuous local search）。

这个模型与前面的离散搜索问题有一个很强的联系。Fearnley、Goldberg、Hollender 和 Savani 发表于 STOC 2021 的工作证明了

$$\boxed{\mathsf{CLS} = \mathsf{PPAD} \cap \mathsf{PLS}.}$$

也就是说，一个搜索问题同时属于 PPAD 和 PLS，当且仅当它能归约到上述连续局部搜索问题。

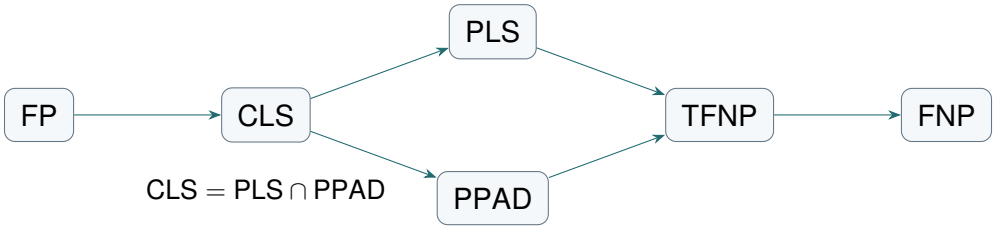


图 12.7 局部搜索相关复杂度类的包含关系，箭头由子类指向包含它的类。

12.7 习题

习题 12.1 (局部最优等于全局最优)**.** 如果一个问题的所有局部最优解都是全局最优解，沿着严格改进的邻居一直搜索，是否就一定能在多项式步内结束？若不能，请构造一族例子，使每个状态都能用 b 位表示，寻找改进邻居和比较目标值都只需关于 b 的多项式时间，但从某个初态出发需要 $2^b - 1$ 次改进才能停止。

习题 12.2 (没有交叉的旅行路线)**.** 若干城市位于平面上，任意三座城市不共线，城市间的距离为欧氏距离。旅行路线中的每条边都画成直线段。

1. 证明：如果一条路线的两条边在各自内部相交，就能通过一次 2-opt 变换严格缩短路线。
2. 反过来，一条没有交叉的路线是否一定是 2-opt 局部最优？考虑五个城市 $a = (0, 0)$，$b = (4, 0)$，$c = (4, 4)$，$d = (0, 4)$，$e = (1, 2)$，检查路线 a, b, e, c, d, a，并说明理由。

习题 12.3 (同时翻转多个顶点)**.** Hopfield 网络的逐点翻转算法一次只改变一个不满意顶点。为了加快搜索，有人建议每轮先按当前构型找出所有不满意顶点，再同时翻转它们。

1. 这种修改是否仍然保证终止？请给出证明或反例。
2. 如果每轮只选出一组两两不相邻的不满意顶点，同时翻转这一组，是否一定终止？要求每轮至少翻转一个点，直到没有不满意顶点为止。请证明你的结论。

习题 12.4 (二分图匹配)**.** 假设有一些学生和一些项目，每个学生可以胜任其中若干个项目。我们希望安排尽可能多的学生参加项目，要求每个学生至多参加一个项目，每个项目也至多接收一个学生。将学生和项目分别作为两组顶点 X 和 Y，学生能够胜任某个项目，就在两者之间连一条边。这样得到的图 $G = (X \cup Y, E)$ 称为二**分图**，它的每条边都连接不同组的两个顶点。一次合法安排对应一组没有公共端点的边，称为一个**匹配**。因此，上面的安排问题就是寻找边数最多的匹配。

1. 从空匹配开始，每次加入一条两端都尚未匹配的边，直到无法继续。证明最终匹配的边数至少为最优值的一半，并给出恰好达到一半的例子。
2. 现在除了直接加边，还允许删去匹配中的一条边，再加入两条边，只要结果仍是匹配。证明：当这两种操作都不能增大匹配时，当前匹配的边数至少为最优值的 $2/3$。

习题 12.5 (平衡两台机器的负载)**.** 有 n 项作业需要交给两台相同机器，作业 i 的处理时间为 $p_i > 0$。记两台机器的总负载为 L_1, L_2，目标是让 $|L_1 - L_2|$ 尽可能小。从任意分配开始，每次把一项作业移到另一台机器，但只接受能严格减小负载差的移动。

1. 假设每项作业的处理时间都不超过所有作业处理时间之和的一半。证明：在任意局部最优分配中，较重机器的负载至多是较轻机器的两倍。
2. 若每次都在能够改进的作业中选择处理时间最长的一项（处理时间相同的作业任选其一），证明每项作业至多移动一次，因此至多移动 n 次。
3. 给出一个满足第一问假设的局部最优分配，使它仍不是全局最优。

习题 12.6 (如果助教可以复制)**.** 期末考试逐渐逼近，有 k 个正在经受 ADS 折磨的同学（编号为 $1,\ldots,k$）希望能找助教补习功课。每个同学 i 都自带一个正整数 w_i，代表他/她学习 ADS 的困难程度，并且这个数字是公开的：每个人不仅知道自己学习的困难程度，也知道别人学习的困难程度。

正在加急完成毕业论文的助教见此情形觉得压力太大，于是复制了 m 个一模一样的自己来辅导同学。助教把这 m 个复制品放在教室后就离开了，留下同学们自由选择 m 个复制品中的一个辅导自己。当然复制品也是一条生命，因此一个复制品的培训效果会随着它辅导的同学的困难程度之和增加而减小，因此每个同学都希望自己选择的辅导助教的困难程度之和尽可能小。于是我们可以定义一个“均衡状态”：在这一状态下，每个同学都选择了一个复制品，且每个同学都不愿意换一个复制品来辅导自己，因为这样一定会使得自己选择的新复制品的困难程度之和大于等于原先的困难程度之和，学习效果不会提升。

注意我们有一个重要的假设。即在选择复制品期间每个同学不能互相讨论，也就是每个人做出的决定都是独立完成的，每个同学只是根据当前状态和自身需求来行动，不能和别人商量着一起做什么。此外，每次只有一个同学更换复制品，更换完成后，下一位同学再根据新的分配作决定。

1. 自然地，可以用局部搜索算法来寻找均衡状态：每个同学首先都任意选择辅导自己的复制品，然后在局部搜索算法的每一步，只要有同学能够通过单独更换复制品改善学习效果，就让其中一个同学更换，再根据新的分配继续检查。只有目标复制品在**接收这位同学之后**的困难程度之和，小于他原先所在复制品的困难程度之和，他才愿意更换。直到没有同学愿意更换时，算法停止。
   (a) 证明：如果算法会停止，那么算法停止的时候，得到的状态一定是一个均衡状态；
   (b) 这一算法对于任意情况都会停止吗？如果是，请给出证明，如果不是，请给出一个反例。
2. 现在放宽假设，假如每个同学在做决策的时候是可以互相交流的，也就是他们可以一起商定最后的选择。这些同学内心的良知告诉他们，他们应该让最后的局势尽可能对每个人公平，也就是希望最小化**困难程度之和最大的复制品的困难程度（简称困难瓶颈）**。
   (a) 最小化困难瓶颈的选择结果一定是一个均衡状态吗？如果是，请给出证明，如果不是，请给出一个反例；
   (b) 若限制 $m=2$，最小化困难瓶颈的选择结果一定是一个均衡状态吗？如果是，请给出证明，如果不是，请给出一个反例；
   (c) 证明：任意一个均衡状态的困难瓶颈一定不大于最小困难瓶颈的两倍。

习题 12.7 (仓库选址的局部搜索)**.** 一家公司要选择若干地点开设仓库，为各个城市供货。候选地点集合为 I，城市集合为 J，均非空。地点 i 的开设成本为 $f_i \geqslant 0$，从 i 向城市 j 供货的运输代价为 $c(i,j) \geqslant 0$。每座城市由已开设仓库中运输代价最低的一处供货。若开设的仓库集合为非空的 $S \subseteq I$，记 $C_f(S)=\sum_{i\in S} f_i$，$C_r(S)=\sum_{j\in J} \min_{i\in S} c(i,j)$，$C(S)=C_f(S)+C_r(S)$。从任意非空集合开始，每次操作允许添加一个仓库、关闭一个仓库，或用一个未开设仓库替换一个已开设仓库。每次操作后至少保留一个仓库，并且只有总成本 $C(S)$ 严格下降时才接受操作，直到无法改进。

1. 设算法停止于 S，最优仓库集合为 S^*。证明 $C_r(S) \leqslant C(S^*)$。提示：把最优方案中由同一仓库供货的城市放在一起，考虑添加这个仓库能够节省多少运输代价。
2. 若把操作限制为只能添加仓库，第一问的不等式是否仍成立？这样的算法是否具有与输入无关的常数近似比？请证明或给出反例。

12.8 参考答案

习题 *12.1.* 不能。取状态 $\{0, 1, \ldots, 2^b - 1\}$，最小化 $f(x) = x$，并规定只有相差 1 的两个状态互为邻居，并从 $2^b - 1$ 出发。 □

习题 *12.2.* 略。 □

习题 *12.3.* 1. 不保证。取三条边权都为 1 的三角形，初态为所有顶点都取 $+1$。此时每个顶点都不满意，同时翻转后得到全 -1，仍然全部不满意。下一轮又回到全 $+1$，于是一直循环。

2. 一定终止。由于本轮翻转的顶点两两不相邻，各次翻转影响的边互不重叠，因此总增量就是各点增量之和，每轮势函数严格增加。

□

习题 *12.4.* 1. 记最终匹配为 M，最优匹配为 M^*。由于 M 中的边没有公共端点，它一共用到了 $2|M|$ 个顶点。M^* 的每条边都至少有一个端点已被 M 匹配，否则就能把它直接加入 M。从 M^* 的每条边中，选出一个已被 M 匹配的端点。这些顶点全部来自前面的 $2|M|$ 个顶点，因此 M^* 的边数至多为 $2|M|$，即 $|M^*| \leqslant 2|M|$。

考虑两名学生甲、乙和两个项目 a, b，甲只能参加项目 a，乙可以参加任意一个项目。如果算法先安排乙参加 a，算法停止，只安排了一名学生。若改为让甲参加 a、乙参加 b，就能安排两名学生。因此，算法得到的匹配大小恰好是最优值的一半。

2. 把 M 与 M^* 的边放在一起，先拿出公共边。其余部分每个顶点至多连接一条 M 的边和一条 M^* 的边，因此图中每个连通部分都是两种边交替出现的路径或环。环中的两种边数相等，路径中的边数至多相差一条。只需考虑 M^* 多一条的路径，设其中有 t 条 M 的边、$t+1$ 条 M^* 的边。如果 $t = 0$，就有一条边可以直接加入 M；如果 $t = 1$，就能删除路径中间的匹配边，加入两端的两条边。这两种情况都不可能，故 $t \geqslant 2$，从而 $t + 1 \leqslant \frac{3}{2}t$。其余路径、环以及公共边也都满足 M^* 边数至多为 M 边数的 $3/2$ 倍。对所有部分相加，得到 $|M^*| \leqslant 3|M|/2$，即所求结论。

□

习题 *12.5.* 记较重机器的负载为 A，较轻机器的负载为 B，差为 $D = A - B$。

1. 在局部最优分配中，较重机器的每项作业都有 $p \geqslant A - B$。如果至少有两项作业，就有 $A \geqslant 2(A - B)$，即 $A \leqslant 2B$。如果只有一项，题设又要求 $A \leqslant (A + B)/2$，所以 $A \leqslant B$。
2. 先考虑较重机器始终没变的一段连续移动，负载差 D 逐步减小，原来因 $p \geqslant D$ 而无法移动的作业，之后仍无法移动。因此，每次都选择最长的可移动作业时，这一段移出的作业只会越来越短或一样长。已经移出的作业位于较轻机器，也没有理由移回来。如果某次移动 p 后，两台机器交换了轻重关系，就有 $2p > D$，新差值为 $D' = 2p - D$。又因为这是一次改进，$p < D$，所以

$D' < p$。这说明刚移过去的作业比新的负载差还长，无法移回，这一段中更早移出的作业至少与它一样长，也都无法移回。以后负载差只会继续减小，它们就永远不会再次移动。

3. 把两项长度为 3 的作业放在第一台，两项长度为 2 的作业放在第二台，已经局部最优。但若每台各分一项 3 和一项 2，就得到更好的 $(5,5)$。

□

习题 *12.6.* 记第 j 个复制品承担的总困难程度为负载 L_j。若同学 i 从 a 转到 b，自己的负担会从 L_a 变成 $L_b + w_i$。因此，他愿意转移的条件是 $L_b + w_i < L_a$。

考虑 1(a) 问，当算法停止时，所有同学都无法通过单独更换改善处境，恰好符合均衡的定义。第 1(b) 问，要证明必定停止，考虑整个分配的负载平方和 $\Psi = \sum_{j=1}^{m} L_j^2$。同学 i 从 a 转到 b 时，

$$\Psi' - \Psi = (L_a - w_i)^2 + (L_b + w_i)^2 - L_a^2 - L_b^2 = 2w_i(L_b + w_i - L_a) < 0.$$

因此，每次个人愿意作出的转移都使同一个势函数严格下降，不会回到旧分配。分配一共只有 m^k 种，算法必定停止。

第 2(a) 问，答案是不一定。取三个复制品和困难程度为 $3, 1, 1$ 的三位同学，让第一位单独使用一个复制品，另两位共用第二个，第三个空着，负载为 $(3, 2, 0)$。最大负载为 3，已经最优，因为困难程度为 3 的同学无论去哪，都会使相应负载至少为 3。但第二个复制品上的任意一位同学都愿意转到空复制品，使自己的负担从 2 降为 1，所以这个最优分配并不均衡。第 2(b) 问，只有两个复制品时一定均衡。若最优分配中还有同学愿意从 a 转到 b，则 $L_b + w_i < L_a$，所以 a 原来较重。转移后的两侧负载分别为 $L_a - w_i$ 和 $L_b + w_i$，都严格小于原先的最大值 L_a，与最优矛盾。第 2(c) 问本质上等价于定理 11.9，不再赘述。 □

习题 *12.7.* 1. 记城市 j 当前的运输代价为 $d_j = \min_{i \in S} c(i, j)$。按最优方案的供货安排，将由仓库 $i \in S^*$ 供货的城市记为 J_i，如果运输代价相同则任选一个仓库。对尚未开设的最优仓库 $i \in S^* \setminus S$，考虑花 f_i 开设它，并让 J_i 中的城市全部改由它供货，其他城市不变。这种安排带来的总成本变化为 $f_i + \sum_{j \in J_i} (c(i, j) - d_j)$。由于算法已经无法通过加仓库改进，而实际为每座城市选择运输代价最低的仓库，所得总成本变化不会超过上述安排，因此这个变化不可能为负。移项得到

$$\sum_{j \in J_i} d_j \leqslant f_i + \sum_{j \in J_i} c(i, j).$$

若 i 已在 S 中，每座城市当前就可以选择它，故 $d_j \leqslant c(i, j)$，上述不等式也成立。对全部 $i \in S^*$ 求和，每座城市恰好计一次，因此

$$C_r(S) \leqslant \sum_{i \in S^*} f_i + \sum_{i \in S^*} \sum_{j \in J_i} c(i, j) = C(S^*).$$

2. 第一问只用到了添加仓库不能改进，所以不等式仍成立。但它只控制运输代价，当前仓库的开设成本仍可能很大。例如，有 m 个候选仓库和一座城市 j，每个仓库的开设成本都为 1，仓库 i 向该城市供货的运输代价为 $c(i, j) = i$，其中 $i = 1, 2, \ldots, m$。若初始就开设全部仓库，只允许添加的算法会立即停止，总成本为 $m + 1$。最优方案只开设编号为 1 的仓库，总成本为 2。比值 $(m + 1)/2$ 可以任意大，因此没有与输入无关的常数近似比。

□

12.9 参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第十二讲。
2. Jon Kleinberg、Éva Tardos，*Algorithm Design*，2006，第 12.1–12.5 节，661–681 页；第 12.7 节，690–699 页。分别介绍局部搜索的基本例子，以及自利决策与势函数。本讲助教问题采用相同机器调度模型。
3. Yurii Nesterov，*Lectures on Convex Optimization*，第 2 版，Springer，2018，第 1.1.3 节。介绍全局优化的均匀网格法及查询复杂度下界，是第 12.6.1 节的参考来源。
4. David S. Johnson、Christos H. Papadimitriou、Mihalis Yannakakis，*How Easy Is Local Search?*，*Journal of Computer and System Sciences*，37(1):79–100，1988；会议版发表于 FOCS 1985，39–42 页。提出了 PLS 复杂度类。
5. Alejandro A. Schäffer、Mihalis Yannakakis，*Simple Local Search Problems that Are Hard to Solve*，*SIAM Journal on Computing*，20(1):56–87，1991。证明稳定构型等问题的 PLS 完全性。
6. John Fearnley、Paul W. Goldberg、Alexandros Hollender、Rahul Savani，*The Complexity of Gradient Descent: CLS = PPAD ∩ PLS*，STOC 2021。讨论连续局部搜索与梯度下降的复杂度。
7. B. W. Kernighan、S. Lin，*An Efficient Heuristic Procedure for Partitioning Graphs*，*Bell System Technical Journal*，49:291–307，1970。介绍保持两侧人数相等的图划分方法。
8. 2024 年诺贝尔物理学奖官方介绍，可了解 Hopfield 网络的背景；Psi4 的分子几何优化文档，可了解通过调整原子位置寻找低能量结构的计算方法。
9. John F. Nash, Jr.，*Equilibrium Points in n-Person Games*，*PNAS*，36(1):48–49，1950。给出有限博弈的混合策略均衡存在性定理。
10. Xi Chen、Xiaotie Deng，*Settling the Complexity of Two-Player Nash Equilibrium*，FOCS 2006，261–270；Xi Chen、Xiaotie Deng、Shang-Hua Teng，*Settling the Complexity of Computing Two-Player Nash Equilibria*，*Journal of the ACM*，56(3)，Article 14，2009。讨论双人均衡的 PPAD 完全性及近似计算的困难性。